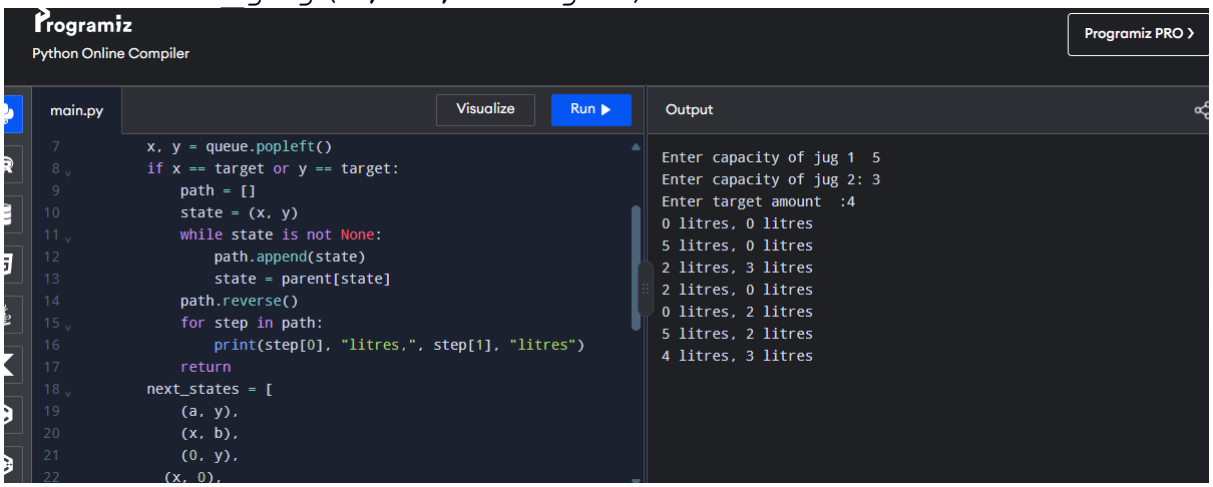
# WATER JUG PROBLEM

1. The target is to measure 4 litres using these operations:

    1. Fill a jug.
    2. Empty a jug.
    3. Transfer water between jugs.

```
from collections import deque
def water_jug(a, b, target):
    queue = deque([(0, 0)])
    visited = {(0, 0)}
    parent = {(0, 0): None}
    while queue:
        x, y = queue.popleft()
        if x == target or y == target:
            path = []
            state = (x, y)
            while state is not None:
                path.append(state)
                state = parent[state]
            path.reverse()
            for step in path:
                print(step[0], "litres,", step[1],
"litres")
            return
        next_states = [
            (a, y),
            (x, b),
            (0, y),
          (x, 0),
            (x - min(x, b-y), y + min(x, b-y)),
            (x + min(y, a-x), y - min(y, a-x))
        ]
        for state in next_states:
            if state not in visited:
                visited.add(state)
```

```
                parent[state] = (x, y)
                queue.append(state)
    print("Target cannot be measured.")
a = int(input("Enter capacity of jug 1  "))
b = int(input("Enter capacity of jug 2: "))
target = int(input("Enter target amount  :"))
if target < 0 or target > max(a, b):
    print("Invalid target amount.")
elif a <= 0 or b <= 0:
    print("Jug capacities must be positive.")
else:
    water_jug(a, b, target)
```




2. There are three jugs with capacities 12 L, 8 L, and 5 L, and the target is 6 L.

The allowed operations are:

1. Empty a jug .
2. Transfer water

```
from collections import deque
def water_jug(capacity, target):
    start = (capacity[0], 0, 0)
    queue = deque([start])
    visited = {start}
    parent = {start: None}
    while queue:
        state = queue.popleft()
```

```
        if target in state:
            path = []
            current = state
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            for step in path:
                print(step)
            return
        next_states = []
        for i in range(3):
            new_state = list(state)
            new_state[i] = 0
            next_states.append(tuple(new_state))
        for i in range(3):
            for j in range(3):
                if i != j:
                    amount = min(
                        state[i],
                        capacity[j] - state[j]
                    )
                    new_state = list(state)
                    new_state[i] -= amount
                    new_state[j] += amount

next_states.append(tuple(new_state))
        for new_state in next_states:
            if new_state not in visited:
                visited.add(new_state)
                parent[new_state] = state
                queue.append(new_state)
a = int(input("Enter capacity of jug 1:"))
b = int(input("Enter capacity of jug 2:"))
c = int(input("Enter capacity of jug 3:"))
target = int(input("Enter target amount:"))
if min(a, b, c) <= 0 or target < 0:
    print("Invalid input.")
elif target > max(a, b, c):
    print("Target exceeds every jug's capacity.")
else:
```

water_jug([a, b, c], target)

